

EETS 7302 TCP/IP Network Administration

Session 9

Linux OS

Some history

- UNIX
 - Thompson & Ritchie, 1969 @BellLabs
 - Re-written in C
 - Multi-user
 - > 45 years old
- UNIX Fragmentation
 - Berkley
 - Novell
 - POSIX
- (in 1990s) Linux • Stallman • Torvalds • Linux open-source, built from scratch to POSIX standard

Why linux ?

- The most important technology for IT/Telecom professionals to understand is Linux.
- Linux is used for more than just traditional server applications. For example, stock market, smart TV , most widely used in smartphone operating system.
- It's providing a foundation for new cloud technologies and container based services as they are emerging in market place.

What makes Linux great?

- Linux is a open source.
- Easy access to a powerful and scriptable CLI.
- Linux is modular all the way down.
- Disc fragmentation is nearly a non-concern. Most (if not all) Linux file systems actively and/or passively avoid such - unlike FAT (and similar), and much better than even NTFS. So no need for that time consuming defrag at all.
- It's not just locked into one vision, or one vendor, or one solution.
- Linux isn't just about one company.
- Linux has been a major source of innovation in IT/Telecom.

Distributions

- RedHat
 - Enterprise
 - RHEL, pronounced “rell”
 - Importance of releases
- CentOS
 - Tons of releases still in use
- Debian
- Suse
- Ubuntu
 - Seems more popular

The kernel and the shell

- The kernel interfaces to the hardware
- The shell is about how the user interacts with the kernel / hardware
- Our shell: bash, the “bourne again shell”
- Other shells
 - bsh - bourne shell
 - ksh- korn shell
- Scripts are written for your specific shell

The shell

- A program that acts as an interface between the user and the OS.
- In linux, we usually mean a command-line shell.
- Shell provides a simple but powerful language to automate and simplify the task that we might want to run on linux system.
- The shell program is replaceable on a user-by-user basis.
- The default shell in linux is bash, the GNU Bourne Again Shell.

sh:

- sh:-The Bourne shell, called "sh," is one of the original shells, developed for Unix computers by Stephen Bourne at AT&T's Bell Labs in 1977. Its long history of use means many software developers are familiar with it. It offers features such as input and output redirection, shell scripting with string and integer variables, and condition testing and looping.

csh and tcsh:

sh and tcsh:

- Developers have written large parts of the Linux operating system in the C and C++ languages. Using C syntax as a model, Bill Joy at Berkeley University developed the "C-shell," csh, in 1978.
- Ken Greer, working at Carnegie-Mellon University, took csh concepts a step forward with a new shell, tcsh, which Linux systems now offer.
- tcsh fixed problems in csh and added command completion, in which the shell makes educated "guesses" as you type, based on your system's directory structure and files.

ksh:

ksh:

- David Korn developed the Korn shell, or ksh, about the time tcsh was introduced. Ksh is compatible with sh and bash.
- ksh improves on the Bourne shell by adding floating-point arithmetic, job control, command aliasing and command completion.
- AT&T held proprietary rights to ksh until 2000, when it became open source.

Bash:

Bash:

- The popularity of sh motivated programmers to develop a shell that was compatible with it, but with several enhancements.
- Linux systems still offer the sh shell, but "bash" -- the "Bourne-again Shell," based on sh -- has become the new default standard.
- One attractive feature of bash is its ability to run sh shell scripts unchanged.

Shell scripts

- Shell scripts are sets of commands that automate programming and maintenance chores; being able to reuse these scripts saves programmers time. Conveniences not present with the original Bourne shell include command completion and a command history.
- It enables easy, local, remote and automated administration.
- The shell is most commonly used to run commands, which consists of up to three parts (1) command itself (2) options that change the behavior of command (3) argument

Linux is Case Sensitive

- Case matters
- In commands
 - e.g., -R is not the same as -r
- In filenames
 - You can have a file named myfile.txt and Myfile.txt in the same directory. They are 2 different files.

ssh

- Secure shell
- Remote login

Some Productivity Enhancers

- <tab> (command and filename completion)
 - [<-Not about saving time; about saving errors]
- up-arrow (command recall)
- | - the pipe for chaining commands
- <> (redirection of input / output)
- ~/ (home directory)

Some quick tips

- . and .. for relative directories
 - . is current directory
 - .. is parent directory
- ; (Semicolon for multiple commands on one line.)
- <ctrl-c> Use at command-prompt.
- <ctrl-z> Use at command-prompt
- man - manual.
 - Available for any command.

clear

- Clear one screen of text
- Every now and then you need a clear screen

history

- See your command history
- Use with grep
- Use with up-arrow
- You can adjust system settings to save more commands
- Stored in ~/.history or ~/.bash_history

pwd - print working directory

- Shows where you are
- Useful in scripts
- You can always use '\$ cd ~/ ' to get back home directory
- Don't type \$, it just represents the command prompt

cd - change directory

- Move around in filesystem
- Use with tab
- With '`$cd -`' you can hop back to the previous working directory.
- Use with `.` and `..`
- Escape special characters (space) in filenames. Tab does this automatically.
- Remember, you can always use '`$ cd ~/'` to get back home

ls - list files

- Look around in filesystem
- Use with tab
- Options
 - l - long format
 - a - all files
 - h - human readable
 - F - Appends indicator so you can more easily spot directories
- Can go multiple directories deep
- Can go “up and over” in directory structure
- I always use ls -al (personal preference)

less - view file

- To browse a file
- Navigation
 - up arrow - one line at a time
 - down arrow - one line at a time
 - b - scroll back a page
 - spacebar - scroll forward a page
 - G - go to end of file
 - g - got to beginning of file
 - / - let's you find text
 - q - quit the program

Wildcards

- *, any characters. Can be one or many.
- ?, any single character

mkdir - make directory

- Can make multiple directories
- Note importance of the / at the beginning (or NOT at the beginning)
- Can make multiple levels deep

```
[smu_user@localhost ~]$ cd Desktop/  
[smu_user@localhost Desktop]$ mkdir mydir  
[smu_user@localhost Desktop]$ cd mydir  
[smu_user@localhost mydir]$ mkdir taskA  
[smu_user@localhost mydir]$ mkdir taskB  
[smu_user@localhost mydir]$ ls -l  
total 0  
drwxrwxr-x. 2 smu_user smu_user 6 Nov 16 03:00 taskA  
drwxrwxr-x. 2 smu_user smu_user 6 Nov 16 03:00 taskB  
[smu_user@localhost mydir]$
```


rmdir - remove directory

- When a directory is empty, you can use **rmdir** to remove the directory.
- And similar to the `mkdir -p` option, you can also use `rmdir` to recursively remove directories.

```
[smu_user@localhost mydir]$ ls
taskA  taskB
[smu_user@localhost mydir]$ rmdir taskA
[smu_user@localhost mydir]$ cd ..
[smu_user@localhost Desktop]$ rmdir mydir
rmdir: failed to remove 'mydir': Directory not empty
[smu_user@localhost Desktop]$
```

rm

- rm stands for remove forever. When you no longer need a file, use rm to remove it.
- Unlike some graphical user interfaces, the command line in general does not have a waste bin or trash can to recover files. When you use rm to remove a file, the file is gone.
- Therefore, be careful when removing files!
- -R Recurse subdirectories
- -i Safety

rm -i

- To prevent yourself from accidentally removing a file, you can type **rm -i**.

```
[smu_user@localhost Desktop]$ ls
file1 file2
[smu_user@localhost Desktop]$ rm -i file1
rm: remove regular empty file 'file1'? y
[smu_user@localhost Desktop]$ rm -i file2
rm: remove regular empty file 'file2'? n
[smu_user@localhost Desktop]$ █
```

Working with file

- Files on Linux (or any Unix) are case sensitive.
- FILE1 is different from file1,
- /etc/hosts is different from /etc/Hosts (the latter one does not exist on a typical Linux computer)

directory

- A directory is a special kind of file, but it is still a (case sensitive!) file.
- Each terminal window (for example `/dev/pts/4`), any hard disk or partition (for example `/dev/sdb1`) and any process are all represented somewhere in the file system as a file.

file utility

- The **file** utility determines the file type. Linux does not use extensions to determine the file type. The command line does not care whether a file ends in .txt or .pdf. As a system administrator, you should use the **file** command to determine the file type.

cp - copy

- To copy a file, use **cp** with a source and a target argument.

```
[smu_user@localhost Desktop]$ ls
file2
[smu_user@localhost Desktop]$ cp file2 file2.copy
[smu_user@localhost Desktop]$ ls
file2  file2.copy
[smu_user@localhost Desktop]$
```

- To copy complete directories, use **cp -r** (the **-r** option forces recursive copying of all files in all subdirectories).

cp - copy

- You can also use `cp` to copy multiple files into a directory. In this case, the last argument must be a directory.

```
[smu_user@localhost Desktop]$ ls
dir2 dir3 file2 file2.copy file3 file4
[smu_user@localhost Desktop]$ ls dir2
file2
[smu_user@localhost Desktop]$ cp file3 file4 dir2/
[smu_user@localhost Desktop]$ ls dir2
file2 file3 file4
[smu_user@localhost Desktop]$
```

- To prevent **cp** from overwriting existing files, use the **-i** (for interactive) option.

mv - move

- mv:
 - Use **mv** to rename a file or to move the file to another directory.
 - When you need to rename only one file then mv is the preferred command to use.
 - The same **mv** command can be used to rename directories.
 - The mv also has a -i switch similar to cp and rm.

```
[smu_user@localhost Desktop]$ ls
dir2 dir3 file2 file2.copy file3 file4
[smu_user@localhost Desktop]$ mv file2 file1
[smu_user@localhost Desktop]$ ls
dir2 dir3 file1 file2.copy file3 file4
[smu_user@localhost Desktop]$
```

head

- You can use **head** to display the first ten lines of a file.

```
[smu_user@localhost etc]$ head passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
[smu_user@localhost etc]$
```

- The **head** command can also display the first **n** lines of a file.

```
[smu_user@localhost etc]$ head -2 passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
[smu_user@localhost etc]$
```

tail:

- Similar to **head**, the **tail** command will display the last ten lines of a file.
- You can give tail the number of lines you want to see.

cat:

- The cat command is one of the most universal tools, yet all it does is copy standard input to standard output.
- use cat to display a file on the screen. If the file is longer than the screen, it will scroll to the end.
- cat is short for concatenate. One of the basic uses of cat is to concatenate files into a bigger (or complete) file.
- For long files, pipe to less

more and less :

- The **more** command is useful for displaying files that take up more than one screen. More will allow you to see the contents of the file page by page. Use the space bar to see the next page, or **q** to quit. Some people prefer the **less** command to **more**.

Create files:

- You can use **cat** to create flat text files. Type the **cat > winter.txt** command as shown in the screenshot below. Then type one or more lines, finishing each line with the enter key. After the last line, type and hold the Control (Ctrl) key and press d.

```
[smu_user@localhost Desktop]$ cat > winter.txt  
It is very cold today!  
[smu_user@localhost Desktop]$ cat winter.txt  
It is very cold today!  
[smu_user@localhost Desktop]$ █
```

tac:

- Just one example will show you the purpose of **tac** (cat backwards).

```
[smu_user@localhost Desktop]$ cat count
one
two
three
four
[smu_user@localhost Desktop]$ tac count
four
three
two
one
[smu_user@localhost Desktop]$
```

grep

- Search within output
- -i, ignore case
- [training@localhost ~]\$ ls Documents/kite-docs-1.0.0/
.....,,, , using..,using.., Using...
- [training@localhost ~]\$ ls Documents/kite-docs-1.0.0/ | grep using
using-kite-properties.html
- [training@localhost ~]\$ ls Documents/kite-docs-1.0.0/ | grep using -i
Using-Kite-with-Apache-Maven.html
Using-the-Kite-CLI-to-Create-a-Dataset.html

find

- Find files
- `$ find . -name *.pdf`
- Can be combined with other commands to search within files.
- Can be combined with other commands to do something, when files are found.
- `[training@localhost ~]$ find Documents/kite-docs-1.0.0/Using*
Documents/kite-docs-1.0.0/Using-Kite-with-Apache-Maven.html
Documents/kite-docs-1.0.0/Using-the-Kite-CLI-to-Create-a-Dataset.html`

touch

- Creates an empty file.
- Updates date/time on files that already exist.

```
[smu_user@localhost Desktop]$ ls -l
total 0
[smu_user@localhost Desktop]$ touch file1
[smu_user@localhost Desktop]$ touch file2
[smu_user@localhost Desktop]$ ls -l
total 0
-rw-rw-r--. 1 smu_user smu_user 0 Nov 16 03:10 file1
-rw-rw-r--. 1 smu_user smu_user 0 Nov 16 03:10 file2
[smu_user@localhost Desktop]$ █
```

chmod - change file permissions

- User, group, other
- Binary
 - 7 = rwx
 - 6 = rw
 - 5 = read and execute
 - 4 = read
- Read, write, execute
- Can be set multiple ways
 - `chmod 755 filename.txt`
 - `chmod u+x filename.txt`
- Execute needed for scripts (and then use `./` if it's in the working directory)
- 777 = Major red flag. File is wide open. Almost never use. Use 755 instead.
- 600 = Very restricted and safe.

nano - text editor

- '\$ yum install nano'
- More user-friendly than vi

vi - text editor

- Most system alias vi to vim. Use vim, if it's available
 - esc, :
 - gg - top of file
 - GG - bottom of file
 - / - search text
 - %s/find/replace/g
 - yy - yank
 - dd - delete
 - Commands can be combined with number for repetition
 - w - write file (save) and stay in editor
 - wq - write file (save) and quit
 - q! - quit without saving
 - esc, :, u - undo

Background/foreground processes

- Use with & to put process into background.
- fg brings it back to the foreground

who

- Who else is logged in to the server
- You can login multiple times
- This allow you to check other users and number of your own sessions

which

- Run to find out which command (actual file) you are executing
- `which <command-name>`

df

- disk free
- Also, like many commands, available in HDFS
- Use -h for “human readable” number

printenv

- Print, to the screen, all environment variables

scp

- Secure copy
- FTP sends credentials in clear text
- `scp -P <port_number> source destination`
- Example from sandbox to laptop:
 - `scp -P 2222 root@11.11.11.11:~/exercise2/somefile.txt .`
 - 2222 is the port number for the sandbox
- Example from laptop to sandbox:
 - `scp -P 2222 somefile.txt root@11.11.11.11:~/`

WC

- Wordcount
- Use with -l to count lines

sudo

- Super user do
- Perform a command as the super-user (root)

BIOS-

BIOS-

- It is a special type of hardware.
- It is the first thing that starts as soon as the computer boots.
- Its main function is to start the boot loaders and also check the working of CPU.

/boot directory-

- This directory contains certain data and files that are required to start the computer or boot the process.
- Kernel
- initrd- Initial RAM disk. It has all that is required to load operating systems.
- Boot loaders.

Kernel modules

Command	Uses of the command
lsmod	Used for listing the loaded kernel modules.
modinfo	Gives information of kernel modules
modprobe	Used for loading kernel modules

file system hierarchy-

Directory	Use
/ (Commonly used)	Root, top of file system hierarchy
/bin (Commonly used)	Binaries or other executable programs(0 or 1)
/boot	Files needed to boot operating system
/dev	Device files controlled by operating system and system administrator
/etc	System configuration files
/home	Home directories like docx, pix, music
/lib, /lib64	System libraries, system libraries 64 bit

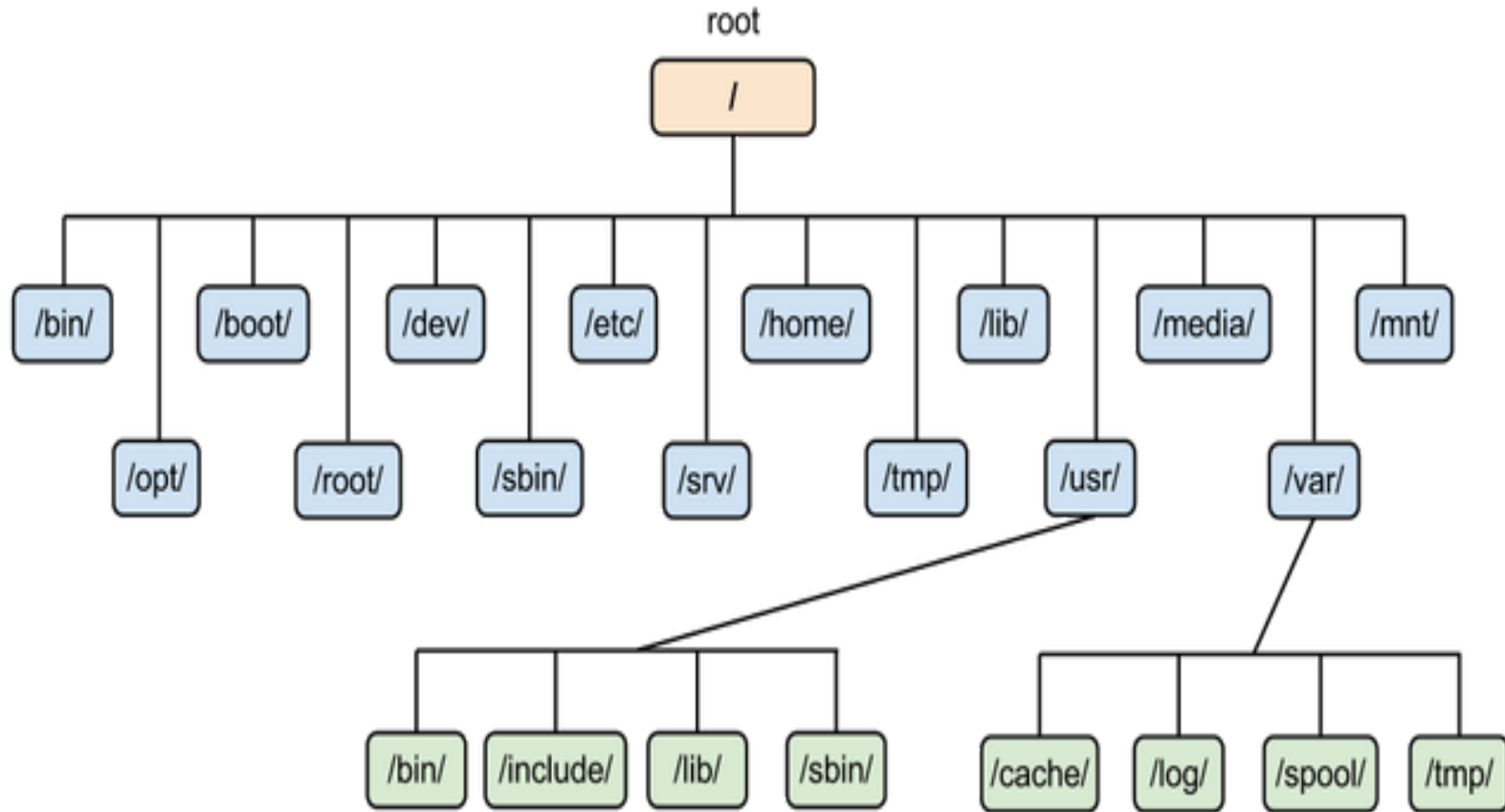
file system hierarchy-

/opt	Optional packages installed on your server
/proc	Provides information of running processes
/root	Home directory for root account
/run	Information created since last boot
/sbin	For system administration commands
/srv	Data served by system or linux server
/sys	Display or configure devices known to linux kernel.

file system hierarchy-

/tmp	Temporary space cleared on reboot
/usr	User related program
/var	Variable data like log files.

file system hierarchy-



End of session